Aplicaciónes Windows Forms

C# Visual Studio

Estructura procedimiento de evento

- **Sender** de tipo Object, tiene una referencia al objeto que lo ha producido
- **e,** clase EventArgs o alguna de sus derivadas con información del evento

```
If (sender Is MiBoton)
        { MessageBox.show (“Mi Boton”); }
else
        { MessageBox.show (“Otro Boton”); }
```

Formularios

- Formularios Windows
  - **Modales**, Un formulario o cuadro de diálogo *modal* debe cerrarse u ocultarse para que el usuario pueda continuar trabajando con el resto de la aplicación. (Foco)
  - **No Modales**, Los formularios *no modales* permiten alternar el foco entre dos formularios sin necesidad de cerrar el formulario inicial(Visible)

Formularios - eventos

_Load: creación del formulario en memoria
Show, showDialog ,
Primera vez que se muestra
Inicialización de variables o entorno
_Activate, recibe foco
_Show

_FormClosing: x cierre form, método .close

```
private void FormClosing (object sender, FormClosingEventArgs e)
{  e.cancel = true; }
```

_FormClosed
_Deactivate: pierde foco

```
public void ExitApplication() {
// Display a message box asking users if they
// want to exit the application.
 if (MessageBox.Show ("Do you want to exit?",
"My Application", MessageBoxButtons.YesNo,
MessageBoxIcon.Question) == DialogResult.Yes)
{ Application.Exit(); }
}
```

Mostrar formularios

- Cuadro dialogo modal

```
Form frmAbout = new Form();
frmAbout.ShowDialog();
```

```
private void mnuAbout_Click(object sender, System.EventArgs e)
        { Form f = new Form();
    f.ShowDialog(this); }       // Establece formulario principal como
                                   propietario.
```

Mostrar formularios

- Cuadro dialogo no modal

```
/Display f as a modeless dialog
Form f= new Form();
f.Show();
```

Si se muestra un formulario de forma modal, el código que sigue al método ShowDialog no se ejecuta hasta que se cierra el cuadro de diálogo. Sin embargo, cuando un formulario se muestra de forma no modal, el código que sigue al método Show se ejecuta inmediatamente después de que se muestre el formulario.

Introducción de datos

```
private void InformationProcessed()
 {
 // This code will set the DialogResult for a form.
 DialogResult = DialogResult.Yes;
// OR
 // This code will set the DialogResult for a button.
button1.DialogResult = DialogResult.No;
}
```

Formularios

- Form1.Hide() ✉visible = False
- Form1.Close()
- Form1.Dispose()

Formularios - variables

Paso de variables

- Variable pública
- (Form2) // Definir método en Form2

```
internal void Ordenes(String Cliente) {
                MessageBox.Show(Cliente); }
```

internal .- método visible por el ensamblado

- (Form1)

```
Form2 frmOrdenes = new Form2();
frmOrdenes.Ordenes("Nombre");
frmOrdenes.Show;
```

Formularios - variables

- Constructor de la clase

```
Form2 ver = new Form2(Texto1,Texto2);
Public Form2(string uno, string dos)
                {}
```

- Sobrecargar el constructor

```
public Form2()
                        {InitializeComponent();}
public Form2(string uno, string dos)
    {InitializeComponent();}
```

Formularios - variables

- Definir método en Form2

(Form1)

```
Form2 ver = new Form2()
ver.variables(string a, int b, float personas())
ver.Show()
```

(Form2)

```
public void variables (string unaA, int otroInt, float personas2())
{  ..... }
*** ya están disponibles las variables en el load
```

Formularios - ejercicios

1) Determinar el orden de ejecución de los eventos relacionados con los formularios **– load – Activate – show – Deativate – closed – closing**

(2) Preparar un formulario que antes del cierre, pregunte si se está seguro del cierre de la aplicación, para cancelar el cierre.

(3) Proyecto que tiene un formulario principal y otro secundario, principal con dos opciones (botones), cita nueva, cita de revisión, ambas se resuelven en el formulario secundario con un array y la cita se guarda según el nº de paciente en un array, donde en cada posición se guarda la fecha de la cita.

Niveles de acceso - Ámbito

Un proyecto es un conjunto de archivos que se compilarán en un **único ensamblado**. (.exe)

public	**Autoriza el acceso a todos los tipos del ensamblado y fuera de el**
private	Sólo para todos los miembros del tipo
internal	Acceso para todos los tipos del ensamblado (solución)
protected	Autoriza el acceso para todos los miembros del tipo heredados de este incluso fuera del ensamblado
protected internal	Acceso para los miembros del tipo o solo para los tipos que heredan de este, en el ensamblado.

Formularios – (Instancia <->clase) clases y miembros estáticos

- Static, permite compartir miembros entre varias instancias y crear clases que no necesiten ser instanciadas.

```
Public class Statica
    {
    public static int i;
    public static void prueba();
    }
```

Un miembro que se marca como static existe una única vez, independientemente del n de instancias de la clase. El miembro compartido se accede mediante el tipo y no a partir de la instancia de la clase.

```
Statica.i
Statica.prueba()
```

Formularios – variables FRM1<->FRM2

- Definir método en Form1

```
Form2 f2 = new Form2(this);
f2.show();
public  void  setFecha(DateTime fecha)
{ mostrar.Text = Convert.ToString(fecha);}
```

- Definir método en Form2

```
 public Form2(Form1 frm)     //Constructor
{ InitializeComponent();  this.frm1 = frm; }
private void confirmar_Click (.....)
{ frm1.setFecha(cita.value);}
```

Formularios – variables

```
// Definición struct
public struct Datos { public string Color;
                      public DateTime Time;
                      public double Altura;
                      public string Categoria; }

//Definición de la lista
List<Datos> DatosList = new List<Datos>();  // Datos homogéneos

//Añadir dos elementos a la lista
DatosList.Add(new Datos() { Color = "Rojo", Time = DateTime.Now,
Altura = 2, Categoria = "categoria" });
DatosList.Add(new Datos() { Color = "Azul", Time = DateTime.Now,
Altura = 5, Categoria = "categoria2" });

//Borrar elemento por indice
DatosList.RemoveAt(0);
```

Formularios

Ejercicio: 4 Formularios (Principal – Registro – Compra – Comprobación)

Gestión de billetes de autobús

1. El formulario principal con 4 opciones (botones)
2. **Registro de nuevos usuarios** con los siguientes DATOS: Nombre - DNI - Tlf. - email (guardar datos validados)
3. **Compra de un nuevo billete** Formulario que solicita el DNI del usuario y si está registrado aparecen sus datos y le **permite** pedir un nº de asiento [Plazas disponibles 1- 60], en el caso de que no esté registrado no puede comprar billetes.
4. La información de los usuarios registrados se guarda en una [Lista/Array], donde se almacenan los datos de cada usuario [nombre,dni,tlf,email,plaza]
5. **Formulario Comprobación**, que muestre la información almacenada en la lista [usuario - plaza]
6. Salir

Validación DNI (función)

```
private void validar_Click(object sender, EventArgs e)
      {  // validar dni
         if (validaDNI(textBox1.Text))
            { MessageBox.Show("DNI válido");  }
         else
            { MessageBox.Show("DNI NO válido"); }}
 private bool validaDNI(string dni)
      {   const string cadena = "TRWAGMYFPDXBNJZSQVHLCKE";
         int numero;
         char letra, letra2;
         // Recupera letra del dni introducido
         letra = Convert.ToChar(dni.Substring(dni.Length-1));
         // Recupera numero y comprueba longitud
         if ((dni.Length != 9) || !int.TryParse(dni.Substring(0, 8), out numero))
         { return false; }
         letra2 = cadena[numero % 23];
         if (letra == letra2)
           {return true;}   else  {return false;}     }
```

Formularios – ArrayList

1) Añadir la referencia a System.Collections;
2) Definir ArrayList, ArrayList usuarios = new ArrayList();

```
ArrayList datos = new ArrayList();  // objetos heterogéneos
List<usuario> datos2 = new List<usuario>();  // objetos homogéneos
```

Propiedades	Métodos
.Capacity	.Add(objet)
.Count	.Clear()
.IsFixedSize	.Clone()
.IsReadOnly	.Contains(object)
usuarios[usuarios.Count -1]	.CopyTo(Array)
	.Insert(Int32,Object)
	.LastIndexOf(object)
	.Remove(object)
	.RemoveAt(int32)
	.Reverse()
	.Sort()

Uso …	Opciones de colección genérica	Opciones de colección no genérica
Almacenar elementos como pares clave/valor para una consulta rápida por clave	Dictionary<TKey,TValue>	Hashtable
Acceso a elementos por índice	List<T>	Array ArrayList
Utilizar elementos FIFO (el primero en entrar es el primero en salir)	Queue<T>	Queue
Utilizar datos LIFO (el último en entrar es el primero en salir)	Stack<T>	Stack
Acceso a elementos de forma secuencial	LinkedList<T>	Sin recomendación
Recibir notificaciones cuando se quitan o se agregan elementos a la colección. (implementa INotifyPropertyChanged y INotifyCollectionChanged	ObservableCollection<T>	Sin recomendación
Una colección ordenada	SortedList<TKey,TValue>	SortedList
Un conjunto de funciones matemáticas	HashSet<T> SortedSet<T>	Sin recomendación